

How the NVDA Data Was Collected, Checked, and Used

A stage-by-stage account of the data workflow behind the NVIDIA analysis — what I did this session, the deterministic pipeline that produced the numbers, and how the data was verified and turned into the long/short call. No Anthropic API key was used for any of it.

Part A — What I actually did this session (the "check for data" step)

#	Action	Tool	Result
1	Mapped the whole repo	file glob	saw the engine, agents, and existing targets/NVDA/
2	Looked for your uploads	listed uploads/, searched for *.pdf/.docx/.xlsx/.csv/.png	only the repo's Amkor placeholder note — no NVDA files → flagged it
3	Checked for an existing data brief	read targets/NVDA/_brief.txt	found it, dated 2026-06-22 (fresh)
4	Read the brief + prior analysis	read fundamentals, market, ratings, news, tech	enough to ground the work
5	Reuse-vs-refresh decision	—	data was current → did NOT re-run the scrapers ; reused the brief as the [FACT] base
6	Grounded the analysis on it	embedded the brief's numbers as a "DATA ANCHOR" in every agent prompt	web used only for [ATTRIBUTED] colour, never to overwrite

The honest headline: I *checked for* existing data, found a fresh local brief, and **reused it rather than re-scraping**. The pipeline below is how that brief was built.

Part B — The deterministic data pipeline (each stage in detail)

run_company.py orchestrates it; every stage is plain-Python, no API key. The pattern throughout: **fetch over plain HTTP (urllib) first, and transparently fall back to a real Chromium browser (Playwright) only if a source bot-blocks** (HTTP 403/429/401/202).

Stage 0 — Target & peer set

target_config.py sets the active target (writes targets/_current.json); peer_map.py maps the AI-compute sector to the peer cohort and regenerates competitors.json (NVDA + AMD, AVGO, MRVL, INTC, TSM, QCOM). Every downstream agent reads that cohort.

Stage 1 — Agent 01 data_agent.py · SEC EDGAR (fundamentals + filings)

- **Sources (no key; SEC only asks for a contact email in the User-Agent):**
- company_tickers.json → ticker → CIK resolution
- data.sec.gov/submissions/CIK#####.json → recent filings (10-K/10-Q/8-K...)
- data.sec.gov/api/xbrl/companyfacts/CIK#####.json → **structured XBRL fundamentals**
- **Throttle:** ≤10 req/s (0.25 s spacing), per SEC's fair-use ask.
- **The clever bit — concept merging.** Issuers change their XBRL tags over time. NVIDIA reported revenue under RevenueFromContractWithCustomerExcludingAssessedTax only through FY2022,

NVDA - Data & Methodology (each stage in detail)

How the data was collected, checked & used · no API key · 2026-06-23

then switched to `Revenues` (carrying FY2024 \$60.9B → FY2026 \$215.9B). The agent **merges the annual series across all candidate concepts (US-GAAP + IFRS)**, highest-priority tag winning on overlap — so the recent years aren't silently truncated.

- **Period hygiene:** keeps only ~full-year periods (350–380 days); latest-filed value wins per fiscal year; balance-sheet items treated as instant snapshots.
- **Derived per year:** revenue & YoY%, gross/operating/net margin, EPS, R&D, operating cash flow, capex, **FCF = OCF – capex**, cash, LT debt, equity → `data/<T>/financials.json`.
- **Auto-validation:** on finish it runs **Agent 01b** `validator_agent.py` to gate the pull.

Stage 2 — Agent 03 `market_agent.py` · prices, cross-checked

- **Two independent sources, on purpose:**
- Yahoo v8 chart (`query1.finance.yahoo.com/v8/finance/chart/<T>`) → price, prev close, day & **52-wk range**, volume, currency, exchange
- CNBC restQuote (`quote.cnbc.com/.../restQuote/...`) → last, prev close, **P/E**, change%
- **Consolidation:** prefers Yahoo, fills gaps from CNBC; the headline **P/E comes from CNBC**.
- **The cross-check:** `price_check()` flags **AGREE if the two prices differ ≤ 2%**, else **MISMATCH**. In the NVDA brief every name shows `cross_source=AGREE` — that's this gate passing.
- **News:** Yahoo RSS headlines, each publisher run through the Source Validator (02b).
- **Auto-validation:** runs **Agent 03b** `market_validator.py` afterward.

Stage 3 — Agent 03c `news_agent.py` · reliable news + sell-side ratings

- **News:** Google News RSS (`news.google.com/rss/search`), up to ~40 items/company; each item's **publisher is validated by 02b** — unreliable outlets are dropped. Items whose titles contain rating language (upgrade/downgrade/price target/initiate...) are flagged as analyst coverage.
- **Sell-side ratings:** **finviz** ratings table scraped via Playwright (US-listed/ADR tickers only) → date | firm | action | rating | price target. This is the source of the "8/8 Buy, median ~\$300, Baird \$500" line in the analysis.

Stage 4 — Agent 02 `tech_agent.py` · reliable-source tech research

- **Discovery (no key):** arXiv API + Crossref API (optionally a Bing sweep via browser), channels **interleaved** so none crowds the others, then de-duplicated by URL.
- **Scrape + gate:** fetch each candidate (`urllib`→browser), then **validate via 02b**; keep only **RELIABLE/BORDERLINE**; sort by tier then score → `data/tech/<topic>/findings.json`.
- **Honest limitation I hit:** for NVDA the scholarly hits were **noisy** (keyword collisions surfaced unrelated physics/accelerator papers). The genuinely on-point ones — CUDA (ACM), CoWoS packaging (IEEE/JIEP), NVLink interconnect (arXiv), RAND on AI power, export-control sources — I used; the rest I down-weighted and leaned on domain knowledge tagged `[ATTRIBUTED]`.

Stage 5 — Agent 02b `source_validator.py` · the reliability scoring (0–100)

Used by Agents 02/03/03c to decide what survives. Scored against `reliable_sources.json`:

Signal	Points
Registry tier 1 (peer-reviewed / academic / standards)	base +60
Registry tier 2 (reputable analysis / journalism)	base +45
Discovered via a scholarly index (arXiv +60 / Crossref +50)	bonus

NVDA - Data & Methodology (each stage in detail)

How the data was collected, checked & used · no API key · 2026-06-23

Trusted TLD (.edu / .gov)	+30
HTTPS	+8
Author byline / publication date / citations-DOI (on fetch)	+10 / +10 / +14
Matches a low-quality signal	-40

Verdict: $\geq 60 \rightarrow$ RELIABLE · $40-59 \rightarrow$ BORDERLINE · $< 40 \rightarrow$ UNRELIABLE (dropped). This is where the [RELIABLE 100] / [BORDERLINE 58] tags in the brief come from.

Stage 6 — Brief assembly · `analyst_common.load_context()`

Stitches everything in `data/` into one **DATA BRIEF**, **scoped** by `active_scope()` to the target + its peers (`company_filter`) and the target's deck-headline topics (`tech_filter`) so one company's run never drags in another's data. Sections, in order: **Company Fundamentals (01)** · **Market Data (03)** · **Analyst Coverage (03c)** · **Tech Research (02)** · **EDGAR Filings (01**, with budgeted 10-K/10-Q text excerpts). That assembled string *is* `targets/NVDA/_brief.txt` — the file I read in Part A.

Part C — How the data was verified and used in the analysis

1. **Reuse, not re-scrape.** The brief was fresh, so I anchored on it rather than re-pulling (which would only have changed numbers under the project's synthetic clock).
2. **One shared fact base.** I embedded the brief's fundamentals/prices/ratings as a fixed "DATA ANCHOR" in every research agent's prompt, with an explicit rule: **do not overwrite anchor financials with web numbers** — keep the model internally consistent.
3. **Confidence tagging on every load-bearing number:** [FACT] (in the brief) · [ESTIMATE] (my model) · [ASSUMPTION] (an input I chose) · [ATTRIBUTED] (third-party / domain knowledge to the Jan-2026 cutoff).
4. **Adversarial verification.** Each thesis's 3 load-bearing claims were attacked from 3 independent lenses (financial / technical / market-structure) to see which survived. That is what killed the bear's "margin structurally peaked" claim and kept its "inference share-leakage" claim alive.
5. **Built-in cross-checks I relied on:** the Yahoo-vs-CNBC `AGREE` price gate, the 02b reliability scores on every news/tech source, and the 01b/03b validators that auto-ran when the data was first collected.

Part D — Data gaps & caveats I surfaced (intellectual honesty)

Gap	Why	How I handled it
NVDA capex / FCF = n/a	XBRL <code>capex</code> concept not matched/extracted for NVDA (tag gap)	used OCF \$102.7B [FACT] ; flagged "pull capex from the 10-K"
32x vs ~43x P/E	CNBC P/E (32.15) is forward/adjusted; trailing on GAAP EPS \$4.90 is ~43x	reconciled the denominator explicitly in the model
TSM in TWD	reporting currency taken from the XBRL unit key	did not compare its absolute revenue to USD peers without $\div 31$
QCOM gross margin missing	<code>gross_profit</code> tag absent in its companyfacts	left blank rather than guess
Scholarly noise	keyword collisions in arXiv/Crossref	used only on-point hits; rest $\rightarrow$ domain knowledge [ATTRIBUTED]
Synthetic mid-2026 clock	this project forward-dates its data	treat prices as as-of 2026-06-22; re-pull before any live use

Part E — How to refresh the data (if you want today's live numbers)

```
python run_company.py NVDA --side auto --no-llm
```

That re-runs Stages 0–6 above (SEC + Yahoo/CNBC + Google News/finviz + arXiv/Crossref, with the Playwright fallback) and rebuilds `targets/NVDA/_brief.txt`. No API key needed. Then the analysis (debate → synthesis → verdict → answers) is redone in-session.

Tags: [FACT]=local data brief · [ESTIMATE]=modeled · [ASSUMPTION]=input · [ATTRIBUTED]=third-party. Not investment advice.